Cátedra de
SISTEMAS OPERATIVOS

UNIDAD 5

ENTRADA-SALIDA

Desarrollo de un driver de caracteres en Linux

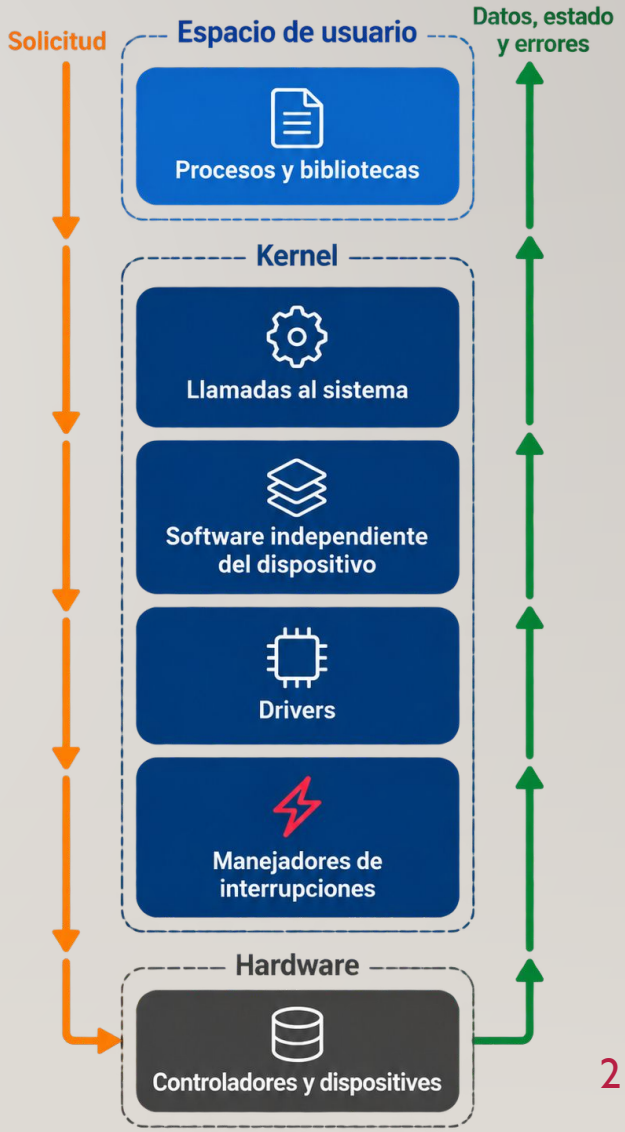
2026

Del proceso de usuario al driver

- Objetivo: recorrer la ruta completa de una operación de E/S
- Producto: un dispositivo virtual funcional en `/dev/sop_char`
- No se requieren conocimientos de programación en C

`cat` → `/dev/sop_char` → `buffer[256]` (dentro del kernel)

- Escritura y lectura de bytes
- Creación automática del nodo en `/dev`
- Registro de eventos y dos comandos `ioctl()` en la solución



Dos espacios, dos niveles de privilegio

Espacio de usuario	Espacio kernel
Procesos aislados	Núcleo y módulos, un único espacio
Fallo afecta al proceso	Fallo puede afectar a todo el sistema
Memoria virtual propia	Acceso privilegiado

- Creación de un módulo que se ejecute en modo kernel
- Utilización de una VM
- Una dirección de memoria de usuario nunca se desreferencia directamente en kernel

Un módulo extiende el kernel en ejecución

- `.ko`: objeto cargable, compilado para un kernel concreto
- `insmod`: inserta un archivo de módulo al kernel
- `modprobe`: resuelve dependencias de módulos
- Un driver integrado y uno modular ofrecen las mismas interfaces internas
- Un módulo no es un "programa normal", no existe `main()` entre otras cosas

`Aplicación` → `syscall` → `VFS` → `file_operations` → `driver` → `device`

- VFS presenta una interfaz uniforme
- El driver implementa las operaciones que admite un dispositivo de caracteres

¿Por qué un dispositivo de caracteres?

- Modelo de flujo de bytes
- Interfaz familiar: `open()`, `read()`, `write()`, `close()`
- Permite concentrarse en VFS sin complejidades como interrupciones, DMA ni buses

```
$ ls -l /dev/sop_char
crw------- 1 root root 240, 0 ... /dev/sop_char
```

- `c`: dispositivo de caracteres, no almacena contenido en un sistema de archivos
- Sus permisos controlan quién puede abrirlo
- major: identifica al driver
- minor: identifica una instancia del dispositivo
- `alloc_chrdev_region()` asigna el número dinámicamente
- `MAJOR(dev_number)` y `MINOR(dev_number)` lo separan

VFS despacha las llamadas

```
open("/dev/sop_char")
read(fd, ...)
write(fd, ...)
```
} `struct file_operations`

- El inode contiene identidad del objeto
- `struct file` representa una apertura concreta y su posición
- `struct file_operations` es la tabla de despacho

```
static const struct file_operations sop_fops = {
    .owner = THIS_MODULE,     .open = sop_open,
    .release = sop_release,   .read = sop_read,
    .write = sop_write,       .unlocked_ioctl = sop_ioctl,
    .llseek = noop_llseek,
};
```

El módulo tiene un ciclo de vida

`insmod` → `sop_init()` → **disponible** → `sop_exit()` ← `rmmod`

- `sop_init()` y `sop_exit()` son los puntos de entrada y salida del módulo
- `MODULE_LICENSE("GPL")` declara la licencia al kernel

```
    ret = alloc_chrdev_region(&sop_dev, 0, 1, "sop_char");

    /* ... */

    unregister_chrdev_region(sop_dev, 1);
```

- Reserva un rango de un dispositivo en `sop_init()`
- Devuelve rango al kernel cuando no es más necesario en `sop_exit()` o en caso de rollback

Conectar cdev y operaciones

```
cdev_init(&sop_cdev, &sop_fops);
sop_cdev.owner = THIS_MODULE;
ret = cdev_add(&sop_cdev, sop_dev, 1);
```

- **cdev** es la representación interna del dispositivo de caracteres
- **cdev_add()** registra el dispositivo de caracteres
- **cdev_del()** deshace el registro del dispositivo

```
sop_class = class_create("sop");
sop_device = device_create(sop_class, NULL, sop_dev, NULL, "sop_char");
```

- **class_create()** crea la clase en **sysfs**
- **device_create()** registra el dispositivo para que **udev** pueda crear **/dev/sop_char**

Operaciones en `struct file_operations`: `open` y `release`

- `open()` prepara el dispositivo para una nueva apertura

```
static int sop_open(struct inode *inode, struct file *file) {
    atomic_inc(&total_opens);
    return 0;
}
```

- Puede eventualmente almacenar estado por apertura en `file->private_data`
- Retorna `0` o un código negativo en caso de error; el `VFS` entrega el descriptor al proceso
- `total_opens` cuenta aperturas acumuladas desde la carga del módulo
- `release()` finaliza una apertura

```
static int sop_release(struct inode *inode, struct file *file) {
    return 0;
}
```

- Se invoca al liberar la última referencia al archivo abierto
- Debe liberar el estado específico de esa apertura

Operaciones en `struct file_operations`: `write`

- `write()` recibe bytes y una posición

```
static ssize_t sop_write(struct file *file, const char __user *src,
                         size_t count, loff_t *pos);
```

- Limitar `count` al espacio disponible
- Actualizar `*pos` y la longitud válida
- Retornar bytes aceptados; `-ENOSPC` si no queda lugar

- Memoria de usuario: cruzar la frontera con cuidado. El puntero está marcado `__user`

```
if( copy_from_user(temporary, src, amount) )
    return -EFAULT;
memcpy(sop_buffer + start, temporary, amount);
```

- `copy_from_user` retorna bytes no copiados; copia primero a un buffer temporal, solo si la copia tiene éxito se modifica el buffer del dispositivo
- `memcpy()` copia entre dos buffers del kernel

✓ Ejemplo simplificado: se omite el manejo del mutex de sincronización

Operaciones en `struct file_operations`: `read`

- `read()` entrega bytes y representa `EOF`

```
static ssize_t sop_read(struct file *file, char __user *dst,
                        size_t count, loff_t *pos);
```

- Retornar `0` representa `EOF`, significa fin de archivo para esa posición
- Una nueva apertura comienza otra vez en posición cero

```
if( *pos < 0 || (size_t) *pos >= data_len)
    return 0;
amount = min(count, data_len - (size_t) *pos);
```

- `copy_to_user()` completa la lectura, devuelve bytes no copiados; `sop_read()` devuelve bytes entregados

```
if( copy_to_user(dst, sop_buffer + *pos, amount) )
    return -EFAULT;
*pos += amount;
return amount;
```

✓ Ejemplo simplificado: se omite el manejo del mutex de sincronización

Operaciones en `struct file_operations`: `ioctl`

- **`ioctl()`** permite ejecutar operaciones que no encajan en **`read()`** o **`write()`**

```
static long sop_ioctl(struct file *file, unsigned int command,
                      unsigned long argument);
```

- Definición de los comandos con macros **`_IO`**, **`_IOR`** o **`_IOW`** en **`sop_char_ioctl.h`**:
  - **`CLEAR`**: borra el buffer bajo el mutex y reinicia posición de la apertura que lo invoca; requiere permiso de escritura
  - **`GET_LEN`**: devuelve **`data_len`** utilizando **`copy_to_user()`**
  - Comando desconocido: devuelve **`-ENOTTY`**

- **`_IOR`**: kernel → usuario
- **`_IOW`**: usuario → kernel
- **`_IO`**: sin datos como argumento

Retornos de `read`/`write` y errores de driver

Retorno	Significado
`> 0`	bytes transferidos en lectura o escritura
`0`	`EOF` en lectura
`-EFAULT`	memoria de usuario inaccesible
`-ENOSPC`	buffer sin espacio
`-ERESTARTSYS`	Espera interrumpida por señal
`-ENOTTY`	Comando de `ioctl()` inválido. "Not a typewriter"

- El kernel utiliza retornos negativos, `libc` expone `-1` y almacena el error en `errno`

El estado compartido requiere sincronización

- `read`, `write`, `ioctl CLEAR` y `GET_LEN` comparten estado:

```
static char sop_buffer[SOP_BUFFER_SIZE];
static size_t data_len;
```

- Es necesario sincronizar el acceso a los datos compartidos

```
static DEFINE_MUTEX(buffer_lock);
…
if( mutex_lock_interruptible(&buffer_lock) )
    return -ERESTARTSYS;
/* leer o modificar buffer y data_len */
mutex_unlock(&buffer_lock);
```

- El mutex puede bloquear y dormir; es apropiado porque estas operaciones se ejecutan en contexto de proceso

Observación del driver sin debugger: `pr_info`

```
    pr_info("sop_char: registrado major=%d minor=%d\n",
        MAJOR(sop_dev), MINOR(sop_dev));
```

- Permite registrar información de diagnóstico
- Hay que evitar registrar cada byte o datos potencialmente sensibles

```
$ sudo dmesg --follow
# o: sudo journalctl -kf
```

- Para ver registros del kernel desde user space

Proyecto mínimo y KBuild

```
driver/
├── Makefile
├── sop_char.c
├── sop_char_ioctl.h
└── test_sop_char.c
```

- El sistema Kbuild (Kernel Build) es el conjunto de scripts y archivos Makefile utilizado para compilar el núcleo de Linux y sus módulos
- Aporta flags y configuración coherentes con el kernel, solo hace falta definir en el Makefile:

```
obj-m += sop_char.o
```

Compilar para el kernel en ejecución

```
$ make -C /lib/modules/$(uname -r)/build M=$PWD modules
```

- Los headers deben coincidir con `uname -r`
- El resultado es el módulo de kernel cargable `sop_char.ko`
- Cualquier warning del compilador merece atención

- El Makefile del proyecto permite ejecutar el comando `make` en el mismo directorio
- Además compila un programa de testeo

Cargar el módulo e inspeccionar

```
$ sudo insmod sop_char.ko
$ lsmod | grep sop_char
$ modinfo sop_char.ko
$ ls -l /dev/sop_char
```

- **`insmod`** carga el módulo y modifica el kernel activo

- Verificación de la inicialización

```
$ sudo dmesg | tail -n 20
$ cat /proc/devices | grep sop_char
$ ls -l /sys/class/sop/sop_char
```

- Mensaje esperado: major/minor asignados
- **`/dev`**, **`/proc`** y **`/sys`** muestran aspectos distintos

Probar desde la terminal

```
$ echo 'Sistemas Operativos' | sudo tee /dev/sop_char >/dev/null
$ sudo cat /dev/sop_char
$ printf '%300s' x | tr ' ' x | sudo tee /dev/sop_char >/dev/null
```

- Prueba caso normal, `EOF` y escritura limitada
- ¿Qué se espera de estos comandos?

```
$ sudo ./test_sop_char
```

- Programa de test, prueba `ioctl`

Descargar módulo: liberar en orden inverso

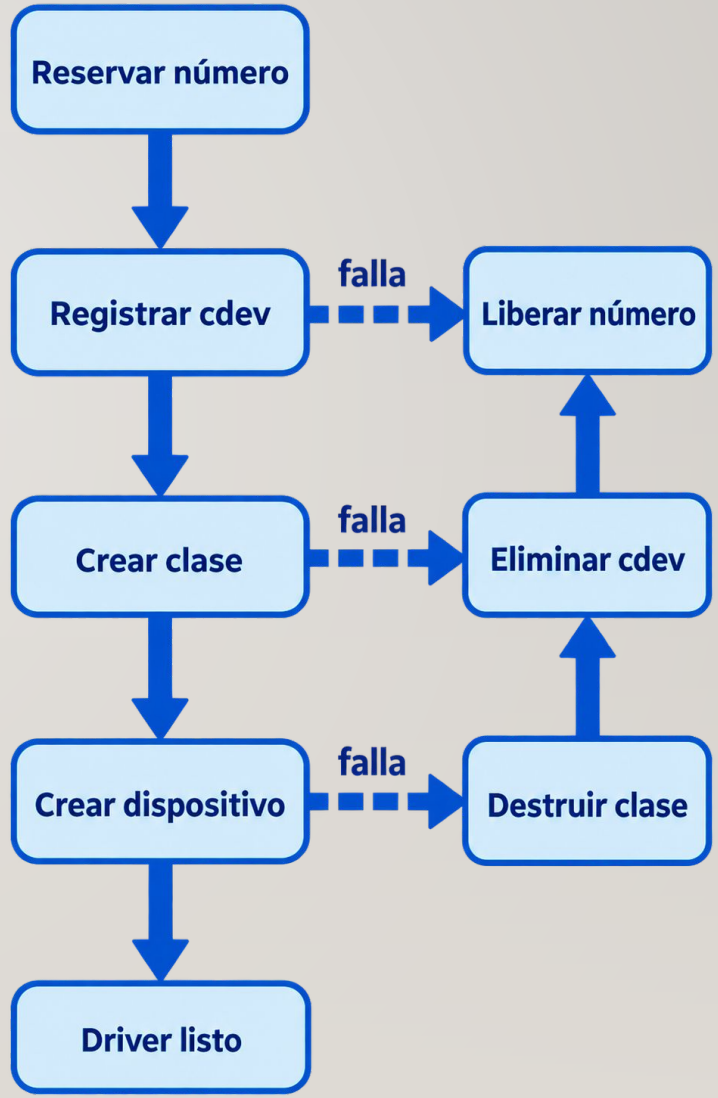
`device_destroy` → `class_destroy` → `cdev_del` → `unregister_chrdev_region`

```
$ sudo rmmod sop_char
$ sudo dmesg | tail -n 20
```

- Confirmar desaparición de `/dev/sop_char`

Rollback en la inicialización

- Lógica de rollback en la inicialización
- Regla central en código de kernel:
  - Cada etapa puede fallar, siempre hay que comprobar posibles errores y la limpieza debe deshacer, en orden inverso todas las operaciones y/o recursos ya creados



¿Dudas o Inquietudes?